Learning the Graph for Relational Deep Learning

Milestone report

Harshal Kulkarni hsk8171@stanford.edu
Project 3: PR to `torch_geometric.contrib` + blog post

1 Problem and paper summary

Relational deep learning [2] turns a database into a graph mechanically: rows become nodes and foreign keys become edges. *What Makes a Desired Graph for Relational Deep Learning?* [1] argues that this schema graph is often poor for message passing: irrelevant columns and tables add noise (*information overload*), and related entities sit many hops apart (*semantic fragmentation*). Its structural optimizer learns the graph together with the model, in four parts: **A** filtering (learned switches on columns and tables, plus a variational bottleneck), **B** injection (new edges from KNN, two-hop, behavioural and temporal templates), **C** gated aggregation (a learned switch per added relation), and **D** an L_0 sparsity penalty. On RelBench [3] it reports, e.g., `driver-top3` ROC-AUC 81.5 (Base) $\rightarrow$ 84.7 (filtering only) $\rightarrow$ 86.8 (full). We implement it as PyG `contrib` modules and test these claims. This milestone covers **A** (switches and penalty) end to end: code, tests and a full experiment.

2 Method: filtering (paper Eq. 3–5, 11)

Base model (RelBench's relational GraphSAGE). Each table's encoder turns row v into $h_v^{(0)}$; two GraphSAGE layers and a linear head give the prediction $\hat{y}_v$.

Gates. Every column c of table T gets a gate $g_{T,c}$ and every table a gate s_T, each with one learned logit θ. In training a gate is a Hard-Concrete sample [4]; at test time it is a hard 0/1 switch (Eq. 4):

$$\text{train: } g = \min\Big(1, \max\big(0,\ \sigma\big(\tfrac{\log u - \log(1-u) + \theta}{t}\big)(\zeta - \gamma) + \gamma\big)\Big), \quad u \sim U(0,1);$$
$$\text{test: } g = \mathbb{I}[\sigma(\theta) > \tau].$$

The gates multiply what the model sees, and the loss pays for every gate left open (Eq. 11):

$$\tilde{e}_{v,c} = g_{T,c}\, e_{v,c}, \qquad \tilde{h}_v^{(0)} = s_T\, h_v^{(0)},$$
$$\mathscr{L} = \mathscr{L}_{\text{task}} + \lambda \sum_{T,c} \mathbb{E}\|g_{T,c}\|_0 + \lambda_s \sum_T \mathbb{E}\|s_T\|_0, \qquad \mathbb{E}\|g\|_0 = \sigma\big(\theta - t \log \tfrac{-\gamma}{\zeta}\big).$$

Here $e_{v,c}$ is column c's 128-d encoding; the column gate acts inside the encoder, before columns are mixed, so a closed column is removed exactly. The paper's Eq. 3 gates the 128 dimensions of $h^{(0)}$ instead; we gate columns so the result is readable ("the model dropped `results.rank`") and document the difference. No table gate is put on the target table. Settings: $\gamma = -0.1$, $\zeta = 1.1$, $t = 2/3$, $\tau = 0.5$.

3 What is built

- **PyG fork** (`contrib`, merged after review): `HardConcreteGate` and `RelationalFilter`, with PyG-style API docstrings (equations, arguments, shapes, examples) and 13 unit tests. Usage, independent of RelBench:

```
filt = RelationalFilter(num_columns={'results': 11, ...}, table_types=[...])
e = filt.gate_columns('results', e)      # [N, C, 128]: column c scaled by g_c
h_dict = filt.gate_tables(h_dict)        # each table's [N, 128] scaled by s_T
col_pen, tab_pen = filt.l0_penalties()   # loss += lambda*col_pen + lambda_s*tab_pen
```

- **Experiment code**: one training script that reproduces RelBench's official example within seed noise; `-model filter`; a resumable sweep runner; a Colab notebook for GPU runs.
- **Correctness checks.** With every gate fixed at 1 the gated model trains *exactly* like Base (same loss at every step, same weights), and over 5 seeds it scores the same ($p = 0.97$). A closed gate removes exactly its column or table: the prediction no longer depends on its values (35 experiment tests in all).
- **Found along the way.** A bug in PyTorch Frame 0.3.0: a missing value gives its column's weights a NaN gradient, so mostly-missing columns stop learning after one step (fixed behind a flag). Encoding each sampled row once per batch makes training 2.5× faster with the same accuracy.

4 Experiment and results

Setup. `rel-f1`, task `driver-top3` (will a driver finish top 3; 1,353 / 588 / 726 train / val / test rows), ROC-AUC. To avoid picking noise, the protocol was fixed in advance: **(A)** tune Base on the paper's grid (learning rate $\{10^{-4}, 10^{-3}, 10^{-2}\}$ × hidden $\{64, 128, 256\}$, 3 seeds); **(B)** tune the filter with the same budget, $\lambda, \lambda_s \in \{10^{-4}, 10^{-3}, 10^{-2}\}$, 3 seeds; **(C)** compare the two winners on 5 fresh seeds and read test once. All runs: 30 epochs, NaN fix on. Gate start $\theta_0 = 5$ and a separate gate learning rate of 0.1 were chosen in 7 dry runs: with the model's learning rate a gate can move only ∼0.3 in the 60–90 training steps this small task allows, and starting at $\theta_0 = 2$ made gates noisy enough to cost ∼0.03 even when none closed.

	Validation (ours)	Test (ours)	Test (paper, Table 1)
Base (lr 10^{-2}, hidden 64)	0.785 ± 0.008	0.766 ± 0.040	0.815 ± 0.011
Base + filtering ($\lambda = \lambda_s = 10^{-4}$)	0.790 ± 0.014	0.772 ± 0.041	0.847 ± 0.006
Difference	$+0.005$ ($p = 0.50$)	$+0.006$ ($p = 0.81$)	$+0.032$

Findings.

- **No measurable gain from filtering** on this task: the paper's +3.2 points is not reproduced. Our test scores vary across seeds (0.72–0.81) far more than the paper reports.
- λ **controls sparsity**, as the paper's Theorem 3.2 needs: by epoch 30, 31, 25 and 13 of 45 columns stay open for $\lambda = 10^{-4}, 10^{-3}, 10^{-2}$. With only 13 open, validation is still within one std of Base: most columns are redundant, but removing them does not help.
- **The gates act too late.** They start closing around epoch 14, while the best validation epoch is around 16, so the selected models are barely filtered (mostly the 3 tables a 2-layer model never reaches). Too fast a gate learning rate closes useful columns instead.
- Our Base is about 5 points below the paper's Base even with the paper's tuning grid.

5 Next steps

(1) Learn the gates first, then freeze them at 0/1 before choosing the final model. **(2)** Injection (KNN, two-hop, behavioural, temporal edges) with gated aggregation (B, C) and the bottleneck (D), then the full optimizer against the paper's 0.868. **(3)** More tasks (`driver-dnf`, `driver-position`, `rel-trial`). **(4)** PR polish and the blog post. Code: PyG fork, branch `contrib/structural-optimizer`; experiments, results and a run-by-run lab notebook: project repository.

References

[1] Y. Cheng and S. Luo. What makes a desired graph for relational deep learning? In *ICML*, 2026. arXiv:2606.08491.

[2] M. Fey, W. Hu, K. Huang, J. E. Lenssen, R. Ranjan, J. Robinson, R. Ying, J. You, and J. Leskovec. Position: Relational deep learning: Graph representation learning on relational databases. In *ICML*, 2024.

[3] J. Robinson, R. Ranjan, W. Hu, K. Huang, J. Han, A. Dobles, M. Fey, J. E. Lenssen, Y. Yuan, Z. Zhang, X. He, and J. Leskovec. RelBench: A benchmark for deep learning on relational databases. In *NeurIPS Datasets and Benchmarks*, 2024.

[4] C. Louizos, M. Welling, and D. P. Kingma. Learning sparse neural networks through L_0 regularization. In *ICLR*, 2018.